useTransition, Step by Step

Keeping typing and clicking responsive while slow rendering happens

Some updates are **urgent**: when you type a letter, it must appear in the box straight away. Others **can wait** a moment, such as redrawing a long filtered list or a heavy tab. Normally React treats every state update the same, so a slow update makes the urgent one wait too, and the page feels sticky.

`useTransition` lets you mark an update as a **transition**, meaning not urgent:

```
const [isPending, startTransition] = useTransition();

startTransition(() => {
  setListQuery(value);   // this update can wait
});
```

React then handles urgent updates first, works on the transition in the background, and abandons it if a newer update makes it out of date. `isPending` is `true` while the transition is still in progress.

This introduction has two examples and one exercise. It is the background for Step 09 of *Movie Night, Part 3*.

> **HOW TO USE THIS HANDOUT**
> Open the `code` folder in VS Code, right-click `index.html` and choose **Open with Live Server**. Every example has a **use startTransition** checkbox, so you can try the same page with and without the transition.

CONTENTS

Part	Topic	Page	Minutes
1	Typing stays smooth while a slow list updates	`01-slow-search.html`	10
2	Switching away from a slow tab	`02-slow-tab.html`	8
3	Exercise: A smooth colour slider	`exercise/starter/`	12
	Total		**30**

Part 1 — Typing stays smooth while a slow list updates

Goal: feel the difference a transition makes on a search box.

Open `01-slow-search.html`.

```
const { useState, useTransition, memo } = React;

const FRUITS = ['apple', 'banana', 'cherry', 'date', 'elderberry', 'fig', 'grape',
'kiwi',
  'lemon', 'mango', 'nectarine', 'orange', 'papaya', 'pear', 'plum', 'raspberry'];
const items = Array.from({ length: 250 }, (_, i) => `${FRUITS[i % FRUITS.length]} $
{i + 1}`);

// Keeps the computer busy for a few milliseconds, standing in for real rendering
work
function busyFor(ms) {
  const start = performance.now();
  while (performance.now() - start < ms) {
    // nothing, on purpose
  }
}

// Each row is slow to render: 250 rows take about a quarter of a second
function SlowItem({ text }) {
  busyFor(1);
  return <li>{text}</li>;
}

// memo: React skips re-rendering SlowList when its props have not changed
const SlowList = memo(function SlowList({ query }) {
  const visible = items.filter((item) => item.includes(query.toLowerCase()));
  return (
    <ul className="list">
      {visible.map((item) => <SlowItem key={item} text={item} />)}
    </ul>
  );
});

function App() {
  const [query, setQuery] = useState('');           // what is in the box: urgent
  const [listQuery, setListQuery] = useState('');   // what the list shows: can wait
  const [transitionOn, setTransitionOn] = useState(true);
  const [isPending, startTransition] = useTransition();

  const handleChange = (e) => {
    const value = e.target.value;
    setQuery(value);                    // always urgent
    if (transitionOn) {
      startTransition(() => {
        setListQuery(value);            // marked as "not urgent"
      });
    } else {
      setListQuery(value);              // urgent too: the box waits for the list
    }
```

```
  };

  return (
    <>
      <div className="row">
        <input type="search" value={query} onChange={handleChange} placeholder="Type
a fruit…" />
        <label>
          <input
            type="checkbox"
            checked={transitionOn}
            onChange={(e) => setTransitionOn(e.target.checked)}
          />
          use startTransition
        </label>
      </div>

      {isPending && <p className="status pending">Updating the list…</p>}
      <div style={{ opacity: isPending ? 0.5 : 1 }}>
        <SlowList query={listQuery} />
      </div>
    </>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

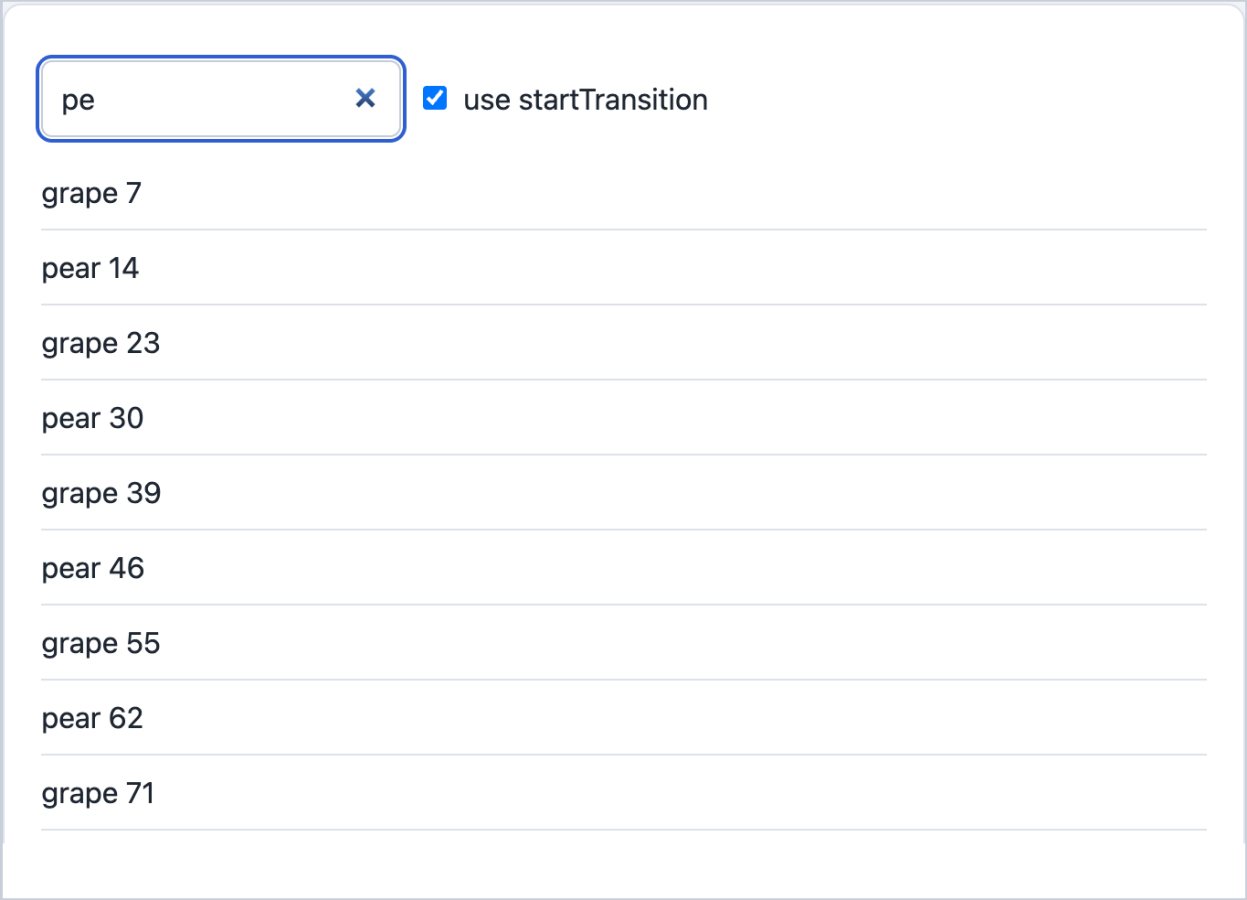


Figure 1 — searching for pe; the list catches up just after the box

Three things make this example work:

1. **The list is slow on purpose.** Each `SlowItem` wastes 1 ms, so all 250 rows take about a quarter of a second to render.
2. **There are two pieces of state for one box.** `query` is what is *in the box* (urgent). `listQuery` is what the *list shows* (can wait). The handler sets `query` normally and sets `listQuery` inside `startTransition`.
3. **`memo(SlowList)`.** `memo` tells React to skip re-rendering `SlowList` when its props are unchanged. Without it, the urgent render (the one that only updates the box) would redraw the slow list too, with the old query, and nothing would be gained.

Try it both ways. Type `p`, then delete it, so that all 250 rows come back:

- **With the checkbox ticked**, the letter appears at once, the old list dims, **Updating the list…** shows, and the full list follows.
- **With the checkbox cleared**, the box freezes until the list has finished. Each keystroke waits for 250 slow rows.

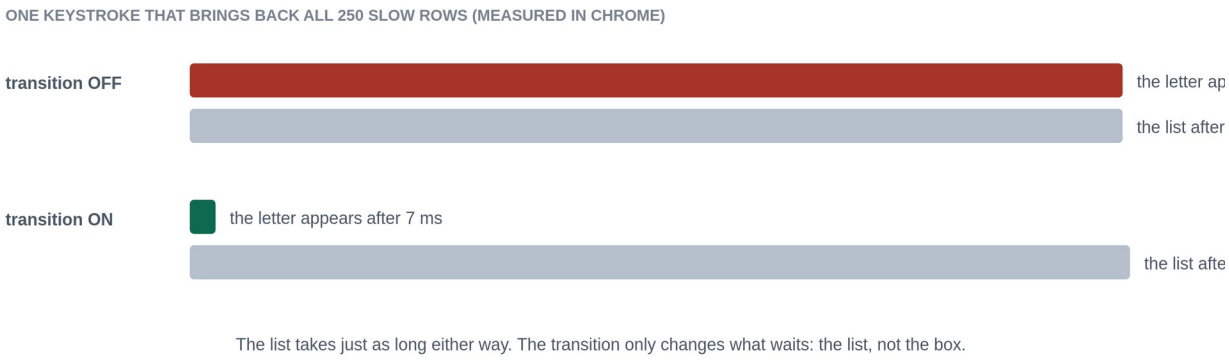


Figure 2 — times measured in Chrome on this page

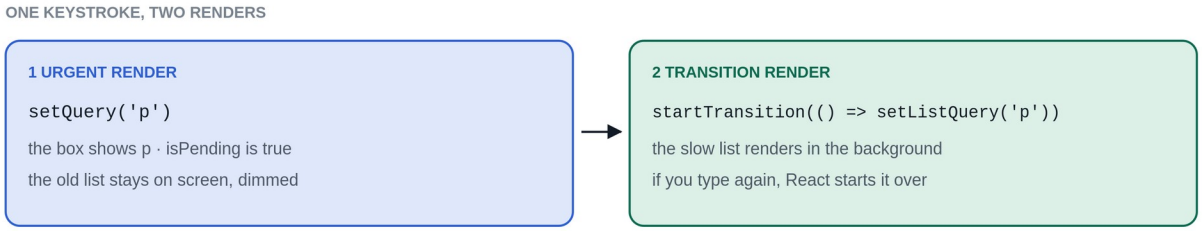


Figure 3 — with a transition, one keystroke causes two renders

---

Part 2 — Switching away from a slow tab

Goal: see React abandon a slow render when a newer update arrives.

Open `02-slow-tab.html`.

```
const { useState, useTransition, memo } = React;

// Keeps the computer busy for a few milliseconds, standing in for real rendering
work
function busyFor(ms) {
  const start = performance.now();
  while (performance.now() - start < ms) {
    // nothing, on purpose
  }
}

function SlowPost({ index }) {
  busyFor(1.5);
  return <li>Post #{index + 1}</li>;
}
```

```
// 300 slow posts: about half a second of rendering
const PostsTab = memo(function PostsTab() {
  const posts = [];
  for (let i = 0; i < 300; i++) {
    posts.push(<SlowPost key={i} index={i} />);
  }
  return <ul className="list">{posts}</ul>;
});

function AboutTab() {
  return <p>Welcome! This tab renders instantly.</p>;
}

function ContactTab() {
  return <p>You can reach us at hello@example.com. This tab renders instantly
too.</p>;
}

function App() {
  const [tab, setTab] = useState('about');
  const [transitionOn, setTransitionOn] = useState(true);
  const [isPending, startTransition] = useTransition();

  const selectTab = (next) => {
    if (transitionOn) {
      startTransition(() => setTab(next));
    } else {
      setTab(next);
    }
  };

  const tabs = [['about', 'About'], ['posts', 'Posts (slow)'], ['contact',
'Contact']];

  return (
    <>
      <div className="row">
        {tabs.map(([id, label]) => (
          <button key={id} className={tab === id ? '' : 'secondary'} onClick={() =>
selectTab(id)}>
            {label}
          </button>
        ))}
        <label>
          <input
            type="checkbox"
            checked={transitionOn}
            onChange={(e) => setTransitionOn(e.target.checked)}
          />
          use startTransition
        </label>
      </div>

      {isPending && <p className="status pending">Loading the tab…</p>}
      <div className="panel" style={{ opacity: isPending ? 0.5 : 1 }}>
        {tab === 'about' && <AboutTab />}
        {tab === 'posts' && <PostsTab />}
        {tab === 'contact' && <ContactTab />}
```

```
      </div>
    </>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

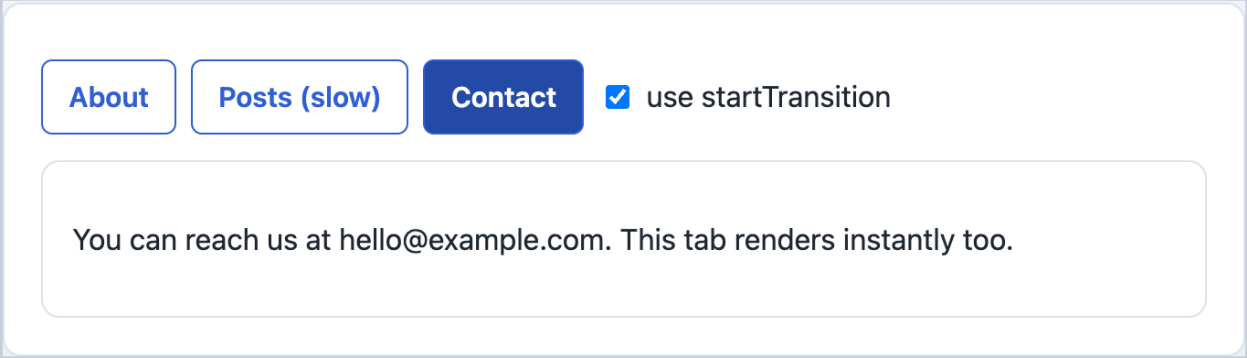


Figure 4 — the Contact tab, reached while Posts was still rendering

1. With **use startTransition** ticked, click **Posts (slow)** and then, straight away, **Contact**. Contact appears almost at once (about 0.15 s in our test). React was rendering Posts in the background, saw a newer update, and dropped the Posts render.
2. Clear the checkbox and do the same. The page is stuck until all 300 posts have rendered (about 0.6 s in our test), and only then switches to Contact.

`isPending` dims the panel and shows **Loading the tab…** while the transition is still running, so the user knows their click was received.

Part 3 — Exercise: A smooth colour slider

Open `exercise/starter/`. Dragging the **Hue** slider recolours a grid of 400 slow swatches, and the slider sticks as you drag. Complete the three TODOs:

1. Split `hue` into two pieces of state: `sliderHue` (what the slider shows: urgent) and `gridHue` (what the grid shows: can wait).
2. Call `useTransition()`, and update `gridHue` inside `startTransition`.
3. While `isPending`, show `<p className="status pending">Repainting…</p>`.

`SlowGrid` is already wrapped in `memo`, so the urgent render skips it.

Check: the slider follows your mouse smoothly, and the grid catches up a moment later.

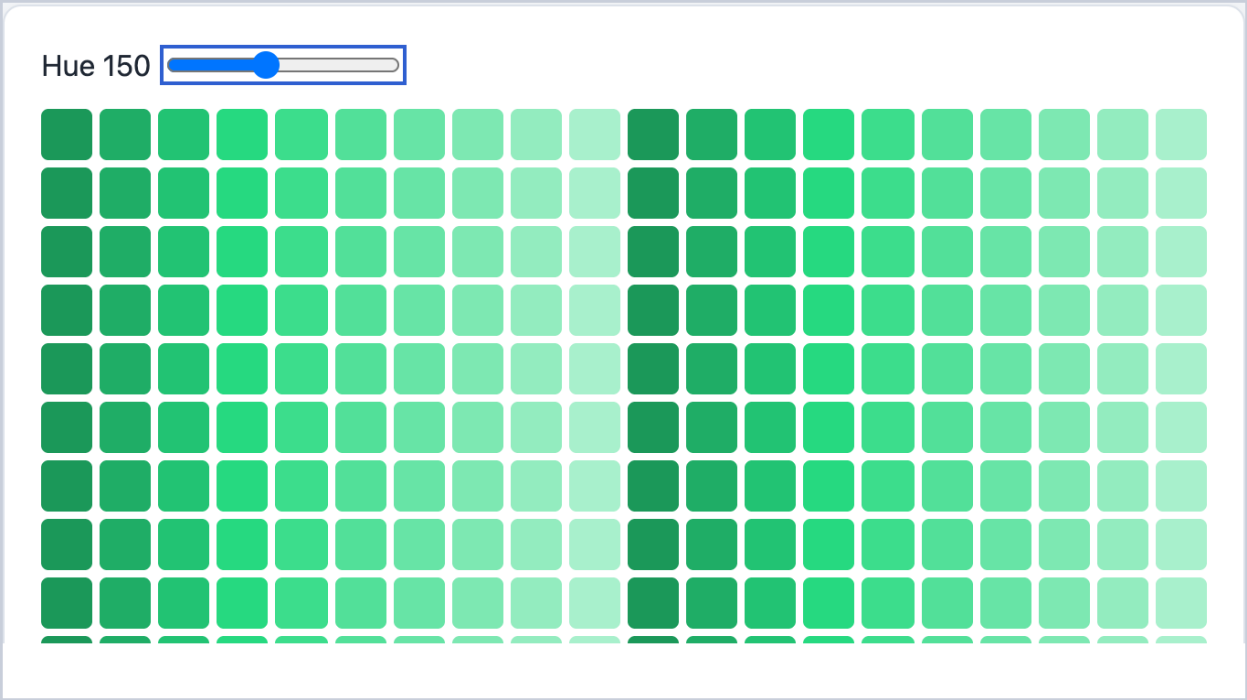


Figure 5 — the finished exercise

A worked solution is in `exercise/solution/app.js`.

Summary

Idea	Code
Get the hook	`const [isPending, startTransition] = useTransition();`
Mark an update as "can wait"	`startTransition(() => setSomething(value));`
Show that the transition is still running	`{isPending && <p>Updating…</p>}`
Keep urgent things urgent	update the input's own value **outside** `startTransition`

Remember: a transition does not make anything faster. The slow work takes just as long; only the *order* changes, with urgent feedback first. Small apps rarely need `useTransition`. It pays off with long lists, heavy tabs and dashboards.

Self-check

- ☐ I can give an example of an urgent update and of one that can wait.
- ☐ I can explain why the search box needs two pieces of state.
- ☐ I know what `isPending` tells me, and how to show it to the user.
- ☐ I know why the input's own update must not go inside `startTransition`.